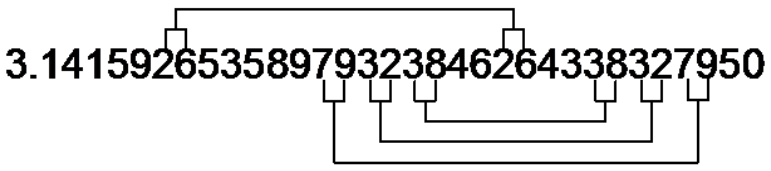
Zipping sequences

Problem: How can we compress objects in practice?



Computer science offers a variety of tools to compress objects, such as the "jpeg" and "png" formats for images; for text or data files, one would use "zip", "bzip2" or "zpaq". The most basic mechanism at work in compressing programs consists of detecting duplicates in the sequence to be compressed.

> **Question:** Does any duplicate pattern give rise to compression?
>
> **Answer:** No. The pattern has to be long enough and the two copies should not be located too far apart.

Can we quantify these conditions?

The program **CopyDetection.py** is able to detect copies within a binary sequence. For instance, it may spot the pattern `0110111001101` of length L = 13 at two different locations of the following string:

```
    0001010110111001101000010111110011011100110110010001011100110001
```

When **the program** finds that a sequence of size L appears at location P_1 and again at location P_2, it refers to the first occurrence instead of repeating the pattern. The second occurrence of `0110111001101` in the above sequence is replaced by `1 1100 1101` where:

- `1` signals the code change,
- `1100` (= 12 in standard binary coding) signals that the original pattern ended 12 bits before (it is smarter to locate the *end* of the previous occurrence), and
- `1101` means that this initial pattern was 13 bits long.

Our code is using space delimiters, as in the Morse code (see Chapter 1).

```
$ CopyDetection.py 000101 0110111001101 000010111110 0110111001101
10010001011100110001
Original:   0001010110111001101000010111110011011100110110010001011100110001 -
length: 64
Encoded:  0001010110111001101000010111110 1 1100 1101 10010001011100110001 -
length: 60
Decoded: 0001010110111001101000010111110011011100110110010001011100110001 -
Correct
```

(for convenience, the program's arguments are joined into a single string).
Note one more time that as explained in Chapter 1, we do not count space characters when computing code length (this choice will be discussed in Chapter 3).

Copy detection 1

1/1 point (ungraded)

Use the program CopyDetection to compress the sequence:

`11100110101001110001100111000011011101001000110001011000001000111100001011001110110100110000011000011`

What is the encoded version of the string?

11100110101001110001 1 1 1001 011011101001000110001011000001000111100001011

✔

Answer: 111001101010011100011 1 1 1001 011011101001000110001011000001000111100001011001110110100110000 1 0 111 11

The program is able to find two duplicated patterns: `100111000` and `0011000`. The

encoded string spares 5 bits.

Submit You have used 1 of 1 attempt

Answers are displayed within the problem

When given no argument, the program `CopyDetection.py` is applied to a pseudo-random binary string of size 100. Make several tries. You should be able to verify that pseudo-random sequences exhibit repeated patterns and thus are, slightly, compressible.

Decimal to binary converter:

Convert

Copy detection 2

0/1 point (ungraded)

Compute the greatest distance between two repeats for which it is advantageous to code the repetition, if the repeated pattern is `0110111001101` .

Note: you have to compute the distance between the onset of the two strings, so don't forget to take the length of the pattern into account.

- 13
- 15
- 64
- 73
- 140 ✔
- 146

✖

Answer
Incorrect:
The code spends 4 bits (`1100`) for the length of the pattern (13), plus 1 signal bit, so the maximal interval length has to be coded with 13 - 5 = 8 bits. If we want a compression by one bit or more, it leaves us with a maximal interval length coded on 7 bits, i.e. 127. So 138 = 127 + 13 is the maximal distance between the two copies.

Submit You have used 1 of 1 attempt

Answers are displayed within the problem

In what follows, $\log_{2+}$ means that the logarithm is rounded to the upper integer.

We approximate the complexity of an integer n as $\log_{2+}(n+1)$.

Copy detection 3

1/1 point (ungraded)

What is the theoretical condition on L (length of the repeated pattern), P_1 (location of the 1st occurrence) and P_2 (location of the 2nd occurrence) to get compression with such a copy detection program?

The constant c represents the offset of the coding scheme (the extra heading `1` in the CopyDetection program).

- (●) $\log_{2+}(P_2 - P_1 - L + 1) < L - \log_{2+}(L + 1) - c$
- ( ) $\log_{2+}(P_2 - P_1 - L + 1) < L - \log_{2+}(P_2 - P_1 + 1) - c$
- ( ) $\log_{2+}(P_2 - P_1 - L + 1) < P_2 - P_1 - \log_{2+}(P_2 - P_1 + 1) - c$

✔

Answer
Correct:
The point is to spare the L bits of the duplicate pattern by echoing a copy located at distance $(P_2 - P_1 - L)$ to the left. So we replace L bits by $\log_{2+}(P_2 - P_1 - L + 1) + \log_{2+}(L + 1) + c$ bits. The condition then reads:

$$\log_{2+}(P_2 - P_1 - L + 1) < L - \log_{2+}(L + 1) - c$$

Submit You have used 1 of 1 attempt

Answers are displayed within the problem

Compression Algorithms in Practice

high frequencies,
which are not perceived well
by the human eye,
are filtered,
which enivitably causes some loss
in the final image.
Conclusion:
Among the various compression
techniques that are used in practice,
many exploit repetitions to code data

‹ Previous Next ›

Legal

Terms of Service & Honor Code
Privacy Policy
Accessibility Policy
Trademark Policy
Sitemap
Cookie Policy
Your Privacy Choices

Connect

Idea Hub
Contact Us
Help Center
Security
Media Kit